

OPERATING SYSTEMS LABORATORY

I/O REDIRECTION USING SYSTEM CALLS

Name	Sathish kumar
Roll Number	2520090008

Task 1: Append Redirection (>>)

Objective: Implement append output redirection using open(), dup(), dup2(), fork(), wait(), and execvp().

Procedure / Commands:

```
nano append_redirect.c

gcc append_redirect.c -o append_redirect
./append_redirect

echo First line >> output.txt
gcc append_redirect.c -o append_redirect
./append_redirect

echo Second line >> output.txt
cat output.txt
```

Output:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with append redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, ">>");

    if (redirect == NULL)
    {
        printf("No append redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 2;

    while (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    int fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd == -1)
```

```

int saved_stdout = dup(STDOUT_FILENO);

if (fork() == 0)
{
    dup2(fd, STDOUT_FILENO);
    close(fd);

    char *args[20];
    int i = 0;

    char *token = strtok(command, " ");

    while (token != NULL && i < 19)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    execvp(args[0], args);

    perror("exec");
    exit(1);
}

wait(NULL);

dup2(saved_stdout, STDOUT_FILENO);
close(saved_stdout);
close(fd);

printf("Append redirection completed successfully.\n");

return 0;
}

```

```

pranay@LAPTOP-OP7R42IK:~$ nano append_redirect.c
pranay@LAPTOP-OP7R42IK:~$ gcc append_redirect.c -o append_redirect
pranay@LAPTOP-OP7R42IK:~$ ./append_redirect
Enter command with append redirection: ./append_redirect
No append redirection found.
pranay@LAPTOP-OP7R42IK:~$ echo First line >> output.txt
pranay@LAPTOP-OP7R42IK:~$ gcc append_redirect.c -o append_redirect
pranay@LAPTOP-OP7R42IK:~$ ./append_redirect
Enter command with append redirection: echo First line >> output.txt
Append redirection completed successfully.
pranay@LAPTOP-OP7R42IK:~$ |

```

```

pranay@LAPTOP-OP7R42IK:~$ cat output.txt
First line
First line
pranay@LAPTOP-OP7R42IK:~$ |

```

Observed output:

```

First line
First line
Second line

```

```

Append redirection completed successfully.

```

Result: The command output is appended to output.txt without overwriting the existing contents.

Task 2: Standard Error Redirection (2>)

Objective: Redirect standard error (stderr) of a command to a file using file descriptors and dup2().

Procedure / Commands:

```
nano stderr_redirect.c

gcc stderr_redirect.c -o stderr_redirect
./stderr_redirect

ls nonexistent 2> error.txt
cat error.txt
```

Output:

```
pranay@LAPTOP-OP7R42IK:~$ ./append_redirect
Enter command with stderr redirection: echo Second line >> output.txt
Append redirection completed successfully.
pranay@LAPTOP-OP7R42IK:~$ cat output.txt
First line
First line
Second line
pranay@LAPTOP-OP7R42IK:~$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with stderr redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, "2>");

    if (redirect == NULL)
    {
        printf("No stderr redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 2;

    while (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
```

```

int saved_stderr = dup(STDERR_FILENO);

if (fork() == 0)
{
    dup2(fd, STDERR_FILENO);
    close(fd);

    char *args[20];
    int i = 0;

    char *token = strtok(command, " ");

    while (token != NULL && i < 19)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    execvp(args[0], args);

    perror("exec");
    exit(1);
}

wait(NULL);

dup2(saved_stderr, STDERR_FILENO);
close(saved_stderr);
close(fd);

printf("stderr redirection completed successfully.\n");

return 0;
}

```

```

pranay@LAPTOP-OP7R42IK:~$ nano stderr_redirect.c
pranay@LAPTOP-OP7R42IK:~$ gcc stderr_redirect.c -o stderr_redirect
pranay@LAPTOP-OP7R42IK:~$ ./stderr_redirect
Enter command with stderr redirection: ls nonexistent 2> error.txt
stderr redirection completed successfully.
pranay@LAPTOP-OP7R42IK:~$ ls nonexistent 2> error.txt
pranay@LAPTOP-OP7R42IK:~$ cat error.txt
ls: cannot access 'nonexistent': No such file or directory
pranay@LAPTOP-OP7R42IK:~$ |

```

Observed output:

```

stderr redirection completed successfully.
ls: cannot access 'nonexistent': No such file or directory

```

Result: The error message generated by the invalid ls command is redirected and stored in error.txt.

Conclusion

The programs demonstrate Linux I/O redirection using system calls. The append redirection program preserves existing file contents and adds new output to the end of the file, while the stderr redirection program sends error messages to a separate file. The screenshots document the source code, compilation, execution, and resulting output.